MailFlow

Complete Work Split — Person 1 & Person 2

Each finishes their full side independently · Then integrate

PS	SIH260461 — Department of Posts — Dynamic Mail Transmission
Rule	Freeze shared-types once (1–2 hrs) → work fully alone → integrate at end
Repo	One monorepo · Person 1 owns api/engine · Person 2 owns web · PRs to main
Tools	Antigravity + AI agents · Paste prompts from this PDF

Person 1 = entire backend brain (routing, API, disruption, learning, alerts, simulation). Person 2 = entire frontend console (Control Tower, Planner, Map, Disruption UI, Tracking, Admin). No day-wise plan. Just finish your list. Integrate when both lists are done.

0. Do This Together First (1–2 hours only)

Create the monorepo and freeze the contract. After this, you never block each other.

Antigravity prompt (run once):

"Create monorepo mailflow with packages/shared-types (TS interfaces: Hub, Leg, Consignment, RouteOption, DisruptionEvent, ReliabilityScore, Booking, Alert, RiskScore), packages/routing-engine (empty computeRoutes), apps/api (Express+TS health), apps/web (Vite React TS Tailwind empty), data/, docs/api.md listing all REST + WebSocket events matching the types. Output folder tree and type files."

Exit: shared-types committed. docs/api.md has every endpoint. Person 1 never touches apps/web. Person 2 never touches apps/api or packages/routing-engine.

Repo structure

```
mailflow/
■■■ packages/shared-types/ ← both freeze once
■■■ packages/routing-engine/ ← PERSON 1 only
■■■ apps/api/ ← PERSON 1 only
■■■ apps/web/ ← PERSON 2 only
■■■ data/ ← PERSON 1
■■■ docs/api.md ← PERSON 1 maintains
■■■ README.md ← both
```

PERSON 1 — Your Complete Work

PERSON 1 · Backend + Routing Engine + Intelligence + Simulation + Alerts

You own these folders only

- packages/routing-engine/
- apps/api/
- data/
- docs/api.md (after freeze)
- scripts/ (seed + sim clock)

Build this complete list (order is suggested, finish all)

#	What to build	Done when
1	Time-expanded graph + Dijkstra routing engine (capacity edges, class weights)	Unit tests pass; top-3 routes + English rationale
2	Seed: 25 Indian hubs + ~100 multimodal legs with schedules & reliability	data/ loads; engine uses it
3	Express API: consignments CRUD + POST /routes/compute + booking confirm	Postman induct → 3 routes → confirm works
4	Disruption service: blast radius + re-solve from current hub + ΔETA	POST /disruptions returns affected + proposals
5	WebSocket: push disruption + position events	Client can subscribe and receive
6	EWMA reliability on leg complete + deadline-risk score	Scores update and affect next routes
7	Resend email on ETA slip + public tracking token	Email fires; token returns tracking data
8	OpenWeatherMap auto-disruption + embargo rules + audit log	Weather/embargo change routing; audit has entries
9	Bulk CSV induction + simulation clock + 1 scripted disruption day	CSV works; full day runs in ~5 min
10	README: how to run API + seed + sim	Other person can start API alone

Copy-paste Antigravity prompts for Person 1

1. "Implement pure TypeScript time-expanded multimodal graph + Dijkstra in packages/routing-engine. Return top-3 RouteOptions with legs, ETA, cost, reliability and English rationale. Class weights: Speed Post prioritizes time, bulk prioritizes cost. Unit tests on 5-hub sample."

2. "Build Express+TS API: consignments, POST /routes/compute, booking confirm, POST /disruptions (blast radius + re-routes with ΔETA), WebSocket events, EWMA reliability, risk score, embargo, audit, bulk CSV, Resend email + tracking token, OpenWeatherMap auto-disruption. Seed 25 hubs + ~100 legs. Simulation clock that runs a full day with one flight cancellation in ~5 minutes."

Person 1 finished when: You can demo the entire backend alone with Postman/curl + sim clock. No frontend needed.

PERSON 2 — Your Complete Work

PERSON 2 · Frontend Console + Map + UX + Demo Polish

You own these folders only

- apps/web/ (entire React app)
- apps/web/src/mocks/ (must match shared-types)

Build this complete list (order is suggested, finish all)

#	What to build	Done when
1	Vite React TS Tailwind shell + dark blue/orange design + sidebar nav	All 7 pages open and navigate
2	Mock layer matching shared-types + api.md	All screens work without real API
3	Route Planner: form → 3 ranked cards + rationale → Confirm	Looks and behaves like final UI on mocks
4	Live Map (Leaflet): hubs, legs by mode, moving markers, disruption pins	Map shows network + filters by mode
5	Control Tower KPIs + activity feed	Numbers + feed update from mocks
6	Disruption Center (blast radius + bulk approve) + What-If form	UI complete; actions hit mocks
7	Consignments register with leg timelines	Search + timeline visible
8	Analytics charts (Recharts) + public tracking page /track/:token	Charts render; tracking page works
9	Admin: embargo UI, bulk CSV upload, audit list	Screens look finished
10	Demo mode + empty/loading/error states	Clean for judges

Copy-paste Antigravity prompts for Person 2

1. "Scaffold Vite React TS Tailwind Framer Motion Router. Dark blue + orange operations console with sidebar: Control Tower, Route Planner, Live Map, Consignments, Disruption Center, Analytics, Admin. Add mocks matching these types: [paste shared-types]. All pages navigate."

2. "Build Route Planner (3 ranked cards + rationale + confirm), Live Map with react-leaflet (hubs, legs by mode, moving markers, disruption pins), Control Tower KPIs + feed, Disruption Center + What-If, Consignments timelines, Recharts analytics, /track/:token page, Admin (embargo, bulk CSV, audit), Demo mode, empty/loading/error states. Everything driven by mocks."

Person 2 finished when: You can demo the entire console alone on mocks. No real API needed yet.

When Both Are Finished — Integrate

Only start this when Person 1 can demo with Postman and Person 2 can demo on mocks.

#	Task	Who
1	Point web from mock URL → real API URL	P2
2	Connect WebSocket for live map + disruptions	P2
3	Fix any type/field mismatches	Both
4	Full loop: induct → routes → confirm → disrupt → re-route → alert → track	Both
5	Run simulation clock end-to-end	Both
6	Deploy (Vercel + Railway/Render + Supabase) + record 5-min demo video	Both
7	Final README with both names as contributors	Both

Hard rules

- Do not change shared-types after freeze without telling the other person.
- Person 1: keep API always runnable with Postman.
- Person 2: never hard-code backend — only go through mocks until integrate.
- Cut everything not on your list (SMS, ML ETA, last-mile, extra polish).
- Push to feature branches often. PR into main. Clear history = clear contribution.

MailFlow · SIH260461 · Person 1 = brain · Person 2 = face · Finish your list · Integrate at the end